

# Lab: Stored XSS into HTML context with nothing encoded

**Platform:** PortSwigger

**Category:** Cross-Site Scripting (XSS)

**Difficulty:** Apprentice

**Tools:** Browser

---

## 1. Objective

The goal of this lab is to exploit a Stored XSS vulnerability in the comment functionality by submitting a comment that executes JavaScript when the blog post is viewed.

---

## 2. Impact

An attacker can inject JavaScript into a comment and store it on the website. When a user views the affected blog post, the injected JavaScript can execute in their browser.

---

## 3. Steps to Solve

Step 1: Find the comment functionality

I opened the lab and browsed the blog post.

I found a comment section where users can submit comments. Since the comment is displayed on the blog page after being submitted, I decided to test whether the application properly handles HTML input.

---

Step 2: Inject the XSS payload

I entered the following payload into the comment field:

- `<script>alert(1)</script>`

The `<script>` tag is used to execute JavaScript, while `alert(1)` displays a simple alert box. I also entered the required name, email, and website fields.

---

Step 3: Submit the comment

I clicked **Post comment** to submit the comment.

The application accepted the comment and stored it on the blog post.

---

Step 4: View the blog post

I went back to the blog post and viewed the page again.

The stored payload was loaded with the comment and executed by the browser.

A JavaScript alert appeared showing: 1

This confirmed that the Stored XSS was successfully exploited

---

## 4. Payload

The payload used was: `<script>alert(1)</script>`

---

## 5. Result

The JavaScript executed successfully when the blog post was viewed.  
The alert box displayed 1, confirming the Stored XSS vulnerability.

---

## 6. Root Cause

The vulnerability exists because the application stores the comment and later renders it in the HTML page without properly encoding the user input.  
As a result, the browser interprets the injected `<script>` tag as executable JavaScript instead of normal text.

---

## 7. Conclusions and Evaluation

What did I learn?

I learned how Stored XSS works when user input is saved by the application and executed later when the affected page is viewed. I also learned how to identify a simple HTML injection point and confirm it using a JavaScript `alert()`.

Lab Rating

4/5 

Did I face any challenges?

The lab was straightforward. The main thing I needed to understand was that the payload was stored with the comment and executed when the blog post was viewed again.

---

## 8. References

- PortSwigger Web Security Academy - Cross-Site Scripting
  - PortSwigger Web Security Academy - Stored XSS
  - PortSwigger Web Security Academy - Stored XSS into HTML context
-

## 8. Screenshots

.....  
want to have more Apps. I want to be able to reply in the postive when I'm asked, 'Are you on?', 'Do you have?' Facebook, YouTube, Messenger, Google search, Google maps, Instagram, Snapchat, Google Play, Gmail and Pandora Radio - to name but a few. And why do I want to have a hundred Apps I'll never use? FOMO.

### Comments

 Zach Ache | 14 August 2026

I was so engrossed in this blog a started squeezing my cat very tightly. I then remembered I don't own a cat' any idea what the number for the pound is?

### Leave a comment

Comment:

`<script>alert(1)</script>`

Name:

cscsdcsdv

Email:

vcvxdx@gmail.com

Website:

https://0a43002804e65cc880de128800210033.web-security-academy.net/post?postId=7

Post Comment

[< Back to Blog](#)



Stored XSS into HTML context with nothing encoded

[Back to lab description >>](#)

LAB Solved 

Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

[Home](#)

Thank you for your comment!

Your comment has been submitted.

[< Back to blog](#)

